Forecasting Privacy-Budget Consumption in Repeated DP-SQL Workloads

CMP653 Term Project — Final Paper

Refik Can Öztaş
Hacettepe University
Ankara, Turkey
N25142279

Abstract

Differentially private (DP) SQL systems add calibrated noise to each aggregate query and charge it a slice of a fixed privacy budget; once the budget is spent, no further query can be answered. They account for queries in isolation, yet real analytical workloads *repeat*—dashboards refresh the same tiles, drill-downs sweep the same query shapes—and an exact repeat can reuse an already-noised answer for free (DP *post-processing*). We give a closed-form model that predicts, from a workload's repetition structure alone and *before any query runs*, how much budget it will spend. To the best of our knowledge, this is the first to forecast such a workload's budget in *closed form* from public repetition structure alone—$\{p_i\}$, length k, budget B—with no query executed and the sensitive data never touched. Reactive caches learn a workload's cost only by running it; the closest proactive method instead precomputes on a *predicted query set* and on the data.

The model's core is the expected number of *distinct* queries a workload contains—the budget is driven by how many distinct queries appear, not how many are issued—from which we derive the expected spend, a savings ratio, and limit, monotonicity, concentration, and utility consequences. We instantiate it in a unit-tested middleware over DuckDB and PostgreSQL (COUNT, SUM, AVG, GROUP BY, and top-k as post-processing of one noised histogram). The GROUP BY / top-k guarantees assume a *public, fully enumerated* key domain—a scope boundary we state up front; private key *discovery* is future work.

On a 4,155-trial campaign (skewed synthetic workloads, TPC-H at two sizes, UCI Adult) the forecast matches within 3% in 21 of 24 cells and is invariant to data size and per-query budget. Our headline result is on *Redbench*—30 workloads derived from real Amazon Redshift traces: forecasting the full workload from only its first half, via an unseen-species correction, cuts the budget-prediction error by 45% ($0.22 \rightarrow 0.12$) over a plug-in estimate. We are explicit about scope: the synthetic grid is a self-consistency check, the temporal (staleness/update) extension is a first-order treatment, and savings vanish on workloads that never repeat—as the model predicts.

1 Introduction and Motivation

Why this matters. Aggregate statistics leak. A pair of COUNT queries that differ by one predicate reveals an individual's attribute by differencing; repeated over a dataset, such queries reconstruct it [3]. Differential privacy (DP) [1, 2] is the standard defense: perturb each answer with noise calibrated to the query's sensitivity and a per-query budget ε_q, and bound the cumulative privacy loss by composition. The budget is a hard, non-renewable resource: once the total $\sum \varepsilon_q$ reaches a cap B, no further query can be answered without breaking the guarantee. A data custodian therefore needs to understand, and ideally predict, *how fast a workload spends its budget.*

A concrete pain. Consider an analytics team that publishes a privacy-protected dashboard: a fixed set of tiles (counts, sums, group-by breakdowns) refreshed on a schedule, plus occasional ad-hoc drill-downs. The custodian sets a monthly budget B and a per-query ε_q. Two failures follow from treating queries in isolation. If ε_q is set to meet an accuracy target, naive composition exhausts B after only B/ε_q releases and the dashboard goes dark mid-month. If ε_q is set small enough to survive the month, every tile is needlessly noisy. The custodian cannot tell, in advance, which will happen—because no model relates the *shape* of the workload (how much it repeats) to the budget it will spend. The repetition is exactly the structure our model exploits: the same dashboard tiles recur, so the budget is governed by the handful of *distinct* templates, not the hundreds of refreshes.

The opening: workloads repeat. Most DP-SQL systems (PINQ [4], Chorus [6], DOP-SQL [7]) charge every query a fresh ε_q and account for them in isolation (PrivateSQL [5] instead amortizes a one-time synopsis budget—Section 2—but still fixes that budget up front rather than forecasting it). Yet analytical workloads are highly repetitive: a monitoring dashboard re-issues the *exact same* KPI tile on a schedule (a canonicalized exact-query repeat, served free), while a drill-down reuses a structural template with *different* literals (a distinct exact query that is charged afresh). Only exact repeats save budget; the savings come from how often a dashboard re-issues identical queries. Under exact-repeat caching, a repeated answer is free—re-using an already-released noisy value adds no privacy cost, since any function of a DP output is itself DP (DP's *post-processing* property)—so the budget a workload of length k consumes is governed not by k but by its number of *distinct* queries u_k (Figure 1). The central question of this project is whether u_k—and hence the budget—can be *predicted from the workload's repetition structure*, in closed form, before the workload runs. We claim no novelty for exact-repeat caching itself—it is a known mechanism; our contribution is *forecasting* its budget impact *before* execution.

Contributions.

(1) **A new capability: a before-execution budget forecast for repeated DP-SQL workloads**—to our knowledge the first that is *closed-form* and reads only public structure ($\{p_i\}, k, B$), never touching the data. *In plain terms: before*

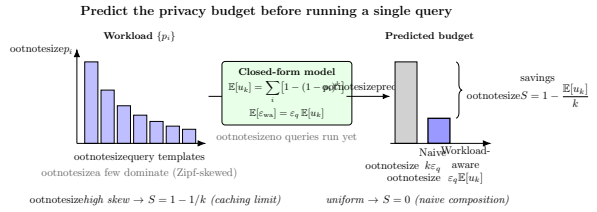

Figure 1: Naive composition charges every one of k queries; workload-aware accounting charges only the u_k distinct query species (an exact query: template *plus* its WHERE literals), since exact repeats are served from cache by post-processing. The model predicts $\mathbb{E}[u_k]$, and hence the budget, before execution.

running anything, the model tells a data custodian how much of the budget the workload will consume and whether it will fit. Concretely, from public workload structure alone—the distinct-query distribution $\{p_i\}$, length k, and budget B, with no query executed—it delivers three *distinct* outputs of deliberately different strengths: (a) the *expected* spend $\varepsilon_q \mathbb{E}[u_k]$, an average-case estimate; (b) with the safe sizing $\varepsilon_q = B/m$, a *hard* completion guarantee, since $u_k \le m$ in the static regime; and (c) a *per-release* (α, β)-accuracy verdict—each release *individually* meets the target with probability $\ge 1 - \beta$, *not* a joint all-succeed guarantee (Section 3). Existing DP-SQL systems do not natively expose such a workload-level forecast (Section 2). Five propositions establish it (expected distinct count, savings, the Zipf-skew case, a concentration bound, and per-query utility), with two limits recovering the folklore $1 - 1/k$ caching rule and standard per-query accounting.

(2) **A measured forecasting improvement on production-derived traces.** On 30 Redbench workloads (synthesized from real Amazon Redshift query traces), forecasting the full distinct-query count from only the *first half* of each workload—via an unseen-species correction—cuts the normalized budget-prediction error by 45% (MAE $0.22 \to 0.12$) over the plug-in estimate (Section 4.2).

(3) **A middleware and a validation campaign.** We instantiate the model over DuckDB and PostgreSQL (exact-repeat caching, a temporal staleness/update extension, and top-k as DP post-processing) and validate the forecast on a 4,155-trial campaign—within $< 3\%$ in $21/24$ cells, invariant to data scale and to ε_q, and exact on a constructed mixed workload—with two leakage experiments confirming the per-query calibration.

We are explicit about scope and do *not* claim to outperform deployed DP systems: the 100× saving over naive composition is a secondary demonstration of the mechanism, the 1.9× edge over a learned bandit is an allocator ablation, and against reactive DP caches (Turbo, CacheDP) we are *on par* on the savings number—the contribution is the before-execution forecast they do not natively expose (Section 4). The headline grid is a self-consistency check (Section 6); the predictive allocator and learned-allocator comparison are in Appendix A, the semantic-caching negative in Appendix D.

2 Related Work

DP-SQL systems. PINQ [4] introduced composable DP query operators; Chorus [6] rewrites SQL to enforce DP; DOP-SQL [7] tunes per-query mechanisms—each exposes a *fixed* per-query ε_q chosen up front and accounts for queries in isolation. PrivateSQL [5] is different in kind: it spends a one-time budget building DP synopses (private materialized views) over a *representative* workload, then answers subsequent queries from those synopses with *no* additional privacy loss—amortization across a declared query set, not per-query accounting. What none of them does is *forecast* a repeated workload's aggregate consumption from its repetition structure: PrivateSQL fixes its synopsis budget up front from a representative set but emits no closed-form, before-execution prediction of $S(k)$ for an arbitrary repeated stream.

Workload-aware DP and caching. The matrix mechanism [10] optimizes noise for a *known* batch of linear queries, and Dong, Sun, and Yi [11] beat worst-case composition by exploiting relational structure. DP caches make repeated work cheaper at runtime: Turbo [13] conserves 1.7–15.9× budget by generalizing across overlapping linear queries, and CacheDP [14] answers a workload at lower budget under an accuracy contract. Two *proactive* points are closest to us. Pioneer [28] reused past noisy results to answer a new query more cheaply (sometimes for free) but, by its authors' own statement, optimizes only the *current* query and leaves future-workload effects to future work; LAPRAS [27] precomputes an offline batch mechanism on a *predicted* query set before the stream arrives and serves it at no extra cost—but it needs a concrete predicted set and *touches the sensitive data* ahead of time, whereas we forecast in closed form from *public* repetition structure ($\{p_i\}, k, B$) alone, the data never touched. These deliver the *savings* our model prices, but reactively (Turbo, CacheDP, Pioneer) or by precomputing on the data (LAPRAS), and without a closed-form structural forecast; our exact-repeat caching is a simple special case and we claim no novelty for the mechanism itself (Section 4).

Multi-query accounting and online allocation. Tight composition—moments accountant [15], Rényi DP [16], zCDP [17]—reduces the *rate* of budget spend and is orthogonal to our *count* of distinct releases. APEx [8] translates a per-query accuracy target into the minimum ε at runtime, one query at a time; BGTplanner [9] learns a Gaussian-process-plus-bandit budget policy for federated-learning rounds, driven by an observed-utility signal. Both decide budget reactively, whereas we forecast it from workload structure. A separate line *schedules* a known budget across competing tasks or analysts—privacy budget scheduling and its fairness variants, and multi-analyst budget sharing [12], which invokes the coupon-collector problem to bound queries-per-analyst (a throughput limit across *analysts*, not a distinct-template forecast). These take per-task demands as input rather than predicting them.

The gap: a before-execution workload forecast. Consider one task: at t=0—before any query runs and before any budget is spent—report, from only public structure ($m, k, \{p_i\}, B$), three *distinct* things: the *expected* spend $\varepsilon_q \mathbb{E}[u_k]$, a *hard* completion guarantee under

Table 1: Positioning. To our knowledge, no prior system natively exposes a closed-form, before-execution budget forecast from public workload structure alone.

System	workload model	predict budget	exact-repeat free
PINQ / Chorus / DOP-SQL	–	no	no
PrivateSQL	repr. batch	no	synopsis
Turbo / CacheDP	–	no	yes (reactive)
Pioneer / LAPRAS	predicted set	no (precompute, on data)	partial
APEx	–	no (per-query)	no
BGTplanner	learned GP	no	no
Ours	**closed-form**	**yes (a-priori)**	**yes**

the B/m sizing ($u_k \le m$), and a *per-release* (α, β)-accuracy verdict (each release individually, not jointly). On this task the established systems do not natively expose an answer: Turbo and CacheDP need cache state and already-executed answers; APEx converts a *single* query's accuracy target into an ε, with no model of workload arrivals or repetition; PrivateSQL builds a synopsis from a representative workload but does not forecast an arbitrary stream's spend; and even the proactive LAPRAS precomputes on a *predicted query set* and on the data, whereas we read only public structure. Our model produces all three directly from $(\{p_i\}, k, B)$, the data never touched—the narrow sense in which it is, to our knowledge, the first. We sit at the intersection of two mature lines—DP-SQL accounting and unseen-species estimation—and borrow the occupancy mathematics and exact-repeat caching, claiming novelty for neither; the contribution is the link between them: a closed-form, before-execution forecast of $S(k)$ from $\{p_i\}$, with the limit, concentration, and utility consequences that follow.

Distinct counts and unseen species. Predicting the number of distinct query species in a length-k workload is the classical unseen-species problem. The occupancy identity $\mathbb{E}[u_k] = \sum_i [1 - (1 - p_i)^k]$ is textbook [18]; horizon-aware extrapolators—Good–Toulmin [19] and its smoothed variant [20]—predict new species from a prefix, while richness estimators such as Chao1 [21] estimate total richness. This line has even been brought *under* DP—INSPECTRE [29] privately estimates the unseen—but for *support coverage*, not budget. We connect these tools to DP-*budget* forecasting; query-template forecasting itself (QueryBot 5000 [22]) is non-private.

The gap. In short, to our knowledge no prior DP-SQL system natively exposes, in closed form and *before execution*, a workload-level forecast from public repetition structure alone—separating, as we do, the *expected* spend, the *hard* completion guarantee of the B/m sizing, and a *per-release* accuracy verdict. Providing that forecast, and measuring on real traces how accurately it can be made from a workload prefix, is the contribution of this project.

3 Methodology

The methodology has three parts, all built on one simple idea. We place a thin *middleware* between the analyst and the database (Section 3.1): every incoming query is checked against the answers already released; an *exact repeat* is returned from cache for free, while a *new* query is given just enough random noise to hide any single row and is charged a slice of the privacy budget. Because identical queries cost nothing, the budget a workload spends is driven by how many *distinct* queries it contains—and hence by how much the workload repeats. The core of the section (Section 3.2) turns this observation into a handful of formulas that predict, *before any query runs*, how many distinct queries a workload will contain, how much budget it will therefore spend, how much that saves over noising every query separately, and how accurate the answers can be at a fixed budget—each with the conditions under which it holds. We then extend the model to data that *changes over time*, where cached answers go stale and must occasionally be refreshed (Section 3.3). Throughout, privacy is never traded away for the forecast: the noise on each released answer stands on its own, and the model only predicts the *bookkeeping*—how fast the budget is used up.

3.1 Threat model and preliminaries

A relational dataset D (DuckDB or PostgreSQL) is queried through the middleware. The adversary is any analyst who can adaptively issue supported queries and observe the noisy outputs; the goal is to bound inference about any single tuple.

Definition 1 (Tuple-level neighbours). *$D \sim D'$ iff they differ in exactly one tuple (unbounded model).*

Definition 2 (ε-DP [1]). *$\mathcal{M}$ is ε-DP iff, for every neighbouring pair D, D' and measurable S,*

$$\mathbb{P}[\mathcal{M}(D) \in S] \le e^{\varepsilon}\, \mathbb{P}[\mathcal{M}(D') \in S].$$

In plain terms. Removing or adding any one person's row may change the *probability* of any output by at most a factor e^{ε}, so no single individual visibly affects the answer: ε is a privacy dial (small ε = more noise, more privacy). The *sensitivity* Δf of a query is how much one row can move its true answer (a COUNT moves by 1, a SUM by the column bound B_c); calibrating the noise to $\Delta f/\varepsilon_q$ is what buys the guarantee. Every query spends ε_q from a fixed budget B; when $\sum \varepsilon_q$ reaches B, no further query can be answered—which is exactly the resource this paper learns to forecast.

Supported SQL and sensitivities. We support SELECT with COUNT, SUM, AVG, single-table FROM, conjunctive WHERE, and optional GROUP BY. Tuple-level sensitivities are $\Delta f = 1$ for COUNT, $\Delta f = B_c$ for SUM(c), and—conservatively—$\Delta f = B_c$ for AVG (a worst-case group of size one, avoiding the implicit n-leak of the empirical-mean mechanism, since n is itself private). The per-column bound B_c is a *public* schema-domain constant (e.g. from the TPC-H spec or the Adult schema); to make the sensitivity hold *by construction* rather than by assumption, the middleware *clamps* each summed/averaged value to $[0, B_c]$ inside the executed SQL (via GREATEST/LEAST, with SQL `NULL`s preserved), so every row's realized contribution is at most B_c even on out-of-domain data. The parser enforces a strict positive whitelist: any construct that could amplify a row's contribution or bypass the sensitivity bound—an aggregate wrapped in arithmetic or a window (e.g. $1000 \cdot$ SUM, COUNT OVER()), CTEs and set operations (a self-union doubles each row), derived tables, OFFSET, and `LIMIT` without an aggregate `ORDER BY`—is rejected *before* any budget is spent.

Laplace release and composition. For sensitivity Δf and budget ε_q the middleware returns $\tilde{f}(D) = f(D) + \eta$, $\eta \sim \mathrm{Lap}(\Delta f/\varepsilon_q)$. A GROUP BY with g groups noises each group independently; since a tuple affects at most one group, *parallel composition* gives total

cost ε_q, not $g\varepsilon_q$. This charges the aggregate *values* only and is ε_q-DP *provided the group-key domain is public and fully enumerated*, so that groups absent from the data are still released with noised zero counts. This is a real requirement, not a formality: returning only the keys *present* in the data—which the current prototype does—leaks membership through a singleton group (a secret degree value present in one record would appear in the output even at small ε). Therefore, all GROUP BY / top-k guarantees in the current prototype assume a *public and fully enumerated* key domain (e.g. the fixed `education` levels, with noised zeros for absent groups). This is a known scope boundary of the prototype, not a defect of the model: lifting it for sensitive or unbounded key domains needs a stability-based threshold that suppresses low-count noisy groups, which we leave to future work. Multi-aggregate queries split ε_q evenly across aggregates by sequential composition. The exact-repeat cache returns a previously released answer with zero budget cost, since it is post-processing.

Top-k as post-processing. The middleware supports top-k (… `GROUP BY` g `ORDER BY agg LIMIT` k) with one essential design point: ordering and truncation act on the *noised* histogram, never the true one. The engine fetches all g groups, noises every count under the same parallel composition (cost ε_q, *not* $k\varepsilon_q$), and only then sorts by the noisy value and keeps the top k. The raw `ORDER BY/LIMIT` is stripped before it reaches the database—letting the DB apply it would select *or order* the reported groups on their *true* counts and leak which groups crossed the cutoff. The same applies to a bare `ORDER BY` *without* a `LIMIT`: it too is ranked on the noised values (true-count row order would otherwise leak the full ranking), and a `LIMIT` *without* an `ORDER BY` is rejected at parse time as an ill-defined selection. Because the released slate and its counts are a deterministic function of the ε_q-DP noisy histogram, top-k is post-processing and charges no extra budget (a report-noisy-max guarantee covering the whole top-k list at the cost of one histogram). The price is that selection is *approximate*: groups whose true counts straddle the k-th boundary may swap. The forecast and cache apply unchanged—a repeated top-k query is one template, free on exact repeat and counted once in $\mathbb{E}[u_k]$.

Assumptions. The forecast assumes (i) a *non-adaptive* analyst whose template choices are independent of the released noise, and (ii) that the template count m and distribution $\{p_i\}$ are public properties of the *query stream*, not of the protected rows. Privacy holds regardless (composition is adaptive-safe and total spend is hard-capped at B); the assumptions concern only the average-case *forecast* (Section 6).

3.2 The analytical model

The budget-spending unit ("species") is the *exact* query—a structural template together with its WHERE literals, i.e. the cache key—since only a *canonicalized exact-query* repeat is served free (the key is the canonical SQL plus typed literals, so it is robust to whitespace and literal-format differences). Let m be the number of distinct exact queries the analyst may issue and $\{p_i\}_{i=1}^{m}$ a distribution over them; the analyst issues k queries i.i.d. from $\{p_i\}$, and u_k is the number of these distinct exact queries that appear at least once. (Two queries sharing a structural template but differing in literals are distinct species and each is charged.)

Proposition 1 (Expected unique queries). $\mathbb{E}[u_k] = \sum_{i=1}^{m} \left[1 - (1-p_i)^k\right]$.

Proof. Let $X_i = \mathbf{1}[\text{template } i \text{ appears}]$. Then $\mathbb{P}[X_i = 0] = (1-p_i)^k$, so $\mathbb{E}[X_i] = 1-(1-p_i)^k$; linearity of expectation gives the result. □

In plain terms. $(1-p_i)^k$ is the chance query i is *never* asked across the k requests; $1-(1-p_i)^k$ is the chance it is asked *at least once*. Adding this up over all m possible queries gives the expected number of *distinct* queries that actually appear—the only ones that ever cost budget. *Worked example (used throughout).* A dashboard with three tiles asked 70%/20%/10% of the time (p=(0.7, 0.2, 0.1)), refreshed k=100 times: each tile is almost surely asked at least once ($1-0.3^{100}$, $1-0.8^{100}$, $1-0.9^{100}$, all ≈ 1), so $\mathbb{E}[u_k] \approx 3$. We pay for about 3 distinct queries, not 100 refreshes—and the formula says so *before* any query runs.

Proposition 2 (Budget consumption and savings). *Naive sequential composition spends* $\varepsilon_{\text{naive}}(k) = k\varepsilon_q$. *Under workload-aware exact-repeat accounting,*

$$\mathbb{E}[\varepsilon_{\text{wa}}(k)] = \varepsilon_q\, \mathbb{E}[u_k],$$
$$S(k) = 1 - \frac{\mathbb{E}[\varepsilon_{\text{wa}}(k)]}{\varepsilon_{\text{naive}}(k)} = 1 - \frac{1}{k}\sum_i \left[1-(1-p_i)^k\right].$$

In plain terms. You pay budget only the *first* time each distinct query is seen, so the saving $S(k)$ is simply the fraction of requests that are exact repeats. In the three-tile example, $S(100) = 1 - 3/100 = 97\%$: ninety-seven of every hundred refreshes are free.

Three limiting cases (the formula at its corners, as sanity checks). *(A) Deterministic repetition* (p_1=1): $S(k) = 1 - 1/k$—the folklore caching rule, now a corner of Proposition 2 at maximum skew, not a standalone claim. *(B) Uniform, $m \to \infty$*: workloads almost never repeat, $\mathbb{E}[u_k] \to k$, $S(k) \to 0$, recovering naive composition. *(C) Uniform, $k \to \infty$, m fixed*: every template eventually appears, so the budget saturates at $m\varepsilon_q$ regardless of k—the practical regime in which a finite-template workload cannot exhaust a budget large enough to cover all m templates.

Proposition 3 (Zipf-parameterized workload). *For $p_i \propto i^{-\alpha}$, the savings ratio $S(k)$ is monotone non-decreasing in α for fixed (k, m). As $\alpha \to \infty$ it approaches Limit A. As $\alpha \to 0$ it approaches the uniform case: naive composition (Limit B)* only when $m \to \infty$*; for finite m with $k \gtrsim m$ the budget instead saturates at $m\varepsilon_q$ (Limit C), which still yields substantial savings $S \approx 1 - m/k$.*

Proof sketch. $\mathbb{E}[u_k] = \sum_i \phi(p_i)$ with $\phi(x) = 1-(1-x)^k$ concave ($\phi'' \le 0$), so $\mathbb{E}[u_k]$ is Schur-concave. Larger α majorizes the weight vector (mass shifts onto top templates), hence lowers $\mathbb{E}[u_k]$ and raises $S(k)$; the endpoints follow from the uniform and point-mass cases. A numerical scan over $m \in \{3, \ldots, 100\}$, $k \in \{2, \ldots, 200\}$ found no violation. □

In plain terms. "Zipf(α)" is a standard knob for how *skewed* a workload is: α=0 means every query is equally likely (no favourites), and larger α piles the requests onto a few popular queries (a steeper "head, long tail"). The proposition says that the more skewed the

workload, the more it repeats and the more budget it saves—the monotone trend we expect. (*Schur-concavity* and *majorization* are just the formal tools that make "more concentrated ⇒ fewer distinct queries" precise.)

Proposition 4 (Concentration of u_k). *Writing $u_k = \sum_i X_i$ with $q_i = 1 - (1 - p_i)^k$, the occupancy indicators are negatively associated, so $\mathrm{Var}[u_k] \le \sum_i q_i(1 - q_i) \le m/4 =: V$, and Chebyshev gives $\mathbb{P}[|u_k - \mathbb{E}[u_k]| \ge t] \le V/t^2$.*

In plain terms. "Concentration" means the budget you *actually* spend on one particular run stays close to the forecast, not merely correct when averaged over many runs—so a custodian can trust the single number. (*Negatively associated* indicators: once one query has appeared, the others are, if anything, slightly *less* likely to also appear, which only tightens the spread around the mean.) Because $V = O(m)$ rather than $O(k)$, the realized u_k has standard deviation $\sqrt{V} = O(\sqrt{m})$ (0.34–1.22 here at m=20): the spread grows only as $\sqrt{m}$, not $\sqrt{k}$. This bounds the *scale* of single-run deviation—a Chebyshev statement, not a tight high-probability guarantee for a given tolerance (a distribution-free McDiarmid bound $2e^{-2t^2/k}$ also holds but is $\sim 18\times$ looser once u_k saturates, so we use it only as a fallback). The budget-exhaustion tail follows: for $c/\varepsilon_q > \mathbb{E}[u_k]$, $\mathbb{P}[u_k\varepsilon_q \ge c] \le V/(c/\varepsilon_q - \mathbb{E}[u_k])^2$.

Proposition 5 (Utility under a fixed total budget). *With a fixed total $\varepsilon_{\mathrm{tot}}$, naive affords $\varepsilon_q^{\mathrm{naive}} = \varepsilon_{\mathrm{tot}}/k$ per query and workload-aware affords $\varepsilon_q^{\mathrm{wa}} = \varepsilon_{\mathrm{tot}}/\mathbb{E}[u_k]$ per unique release; since $\mathbb{E}[|\eta|] = \Delta f/\varepsilon_q$, the predicted per-query error ratio is $k/\mathbb{E}[u_k]$.*

In plain terms. With a fixed total budget, naive accounting must spread it thinly across all k requests, whereas workload-aware accounting spreads it across only the $\mathbb{E}[u_k]$ *distinct* ones—so each released answer gets a $k/\mathbb{E}[u_k]\times$ bigger slice of budget and proportionally less noise. In the three-tile example (k=100, $\mathbb{E}[u_k]\approx$3) that is up to $\approx 33\times$ less error per answer at the *same* privacy cost (in expectation; see the caveat below on capping spend when the realized u_k exceeds $\mathbb{E}[u_k]$).
At Zipf α=1, m=10, k=100, $\mathbb{E}[u_k] \approx 9.9$, so workload-aware accounting answers the same workload at roughly $10\times$ lower per-query error at equal total budget. Two honest caveats. First, the sizing $\varepsilon_q^{\mathrm{wa}} = \varepsilon_{\mathrm{tot}}/\mathbb{E}[u_k]$ targets the *expected* budget: since the realized u_k can exceed $\mathbb{E}[u_k]$, an allocator must cap per-query spend at the remaining budget (and reject once exhausted) to enforce a hard B—which is why the static-regime safe choice is $\varepsilon_q = B/m$ ($u_k \le m$, never overruns). Second, a cached answer reuses a *single* noise draw, forgoing the variance reduction of k independent draws—the budget saving and the lost averaging are two sides of the same coin.

3.3 System and temporal extension

The middleware is a thin proxy between the analyst and a backend database (Figure 2). Each query is parsed and validated, hashed to a (template, parameter) key, and checked against a two-level cache; a hit is returned for free (post-processing), a miss is charged ε_q, noised, and cached. Four core execution modes (exact, naive-DP, workload-aware-DP, temporal-DP) share the pipeline of Algorithm 1—two further modes extend it (predictive allocation, Appendix A; semantic caching, Appendix D)—which couples cache validity to a temporal regime: it advances a logical clock, simulates per-step

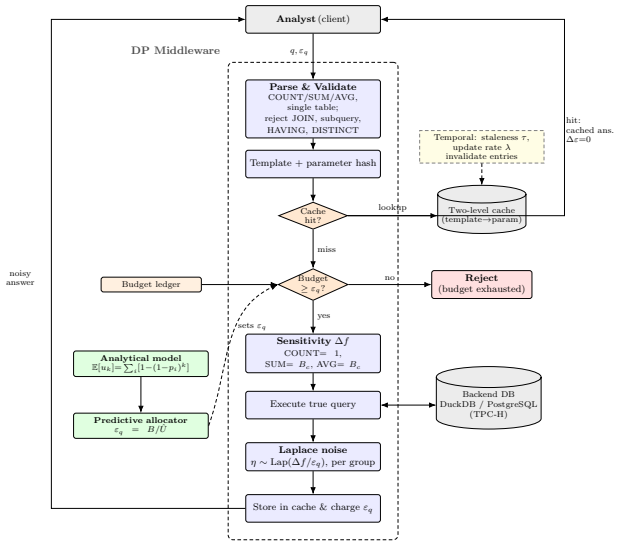

Figure 2: Workload-aware DP-SQL middleware. A cache hit is free by post-processing; a miss is charged ε_q, noised, and cached. The analytical model drives the predictive allocator (ε_q=$B/\hat{U}$), and a temporal regime (τ, λ) invalidates stale entries.

Algorithm 1 Workload-aware DP middleware (one query).

Require: Query q, budget ε_q, ledger $\mathcal{L}$, staleness τ, update $(\lambda, q_{\mathrm{inv}})$

Parse/validate q; advance clock; invalidate each entry w.p. λq_{inv}
Compute template hash h_T, parameter hash h_P
if $\mathcal{L}$.cache$[h_T][h_P]$ fresh (age $\le \tau$) **then**
  return cached $\tilde{r}$ (post-processing, $\Delta\varepsilon$=0)
end if
if $\mathcal{L}$.remaining $< \varepsilon_q$ **then**
  return BudgetExhausted
end if
$\Delta f \leftarrow$ sensitivity; $\mathcal{L}$.consumed += ε_q
$r \leftarrow$ execute q; $\tilde{r} \leftarrow r + \mathrm{Lap}(\Delta f/\varepsilon_q)$ per aggregate, per group
Cache and **return** $\tilde{r}$

Bernoulli invalidations (each entry invalidated independently w.p. λq_{inv}, a discrete approximation of a Poisson process), ages each entry against the staleness tolerance τ, and only then returns the cached release or charges the ledger.

A *temporal extension* couples budget to time, addressing the milestone note that "budget and temporality should be considered together": data updates invalidate cached answers and freshness requirements force re-noising. We present this as a deliberately *first-order* treatment—a planning estimate rather than a full model—and a complete staleness/update theory (with re-release policies and a proper arrival process) is a study in its own right, which we leave to future work.

Table 2: Model vs. empirical u_k, Zipf (m=7, 30 trials/cell). Parentheses: signed relative error (%).

$\alpha \backslash k$	10	25	50	100
0.0	5.50/5.53 (−0.6)	6.85/6.77 (1.2)	7.00/7.00 (0.0)	7.00/7.00 (0.0)
0.5	5.30/5.23 (1.3)	6.73/6.73 (−0.1)	6.98/6.93 (0.7)	7.00/7.00 (0.0)
1.0	4.73/4.70 (0.6)	6.32/6.30 (0.3)	6.88/6.90 (−0.3)	7.00/7.00 (0.0)
1.5	3.98/3.87 (2.8)	5.57/5.70 (−2.3)	6.48/6.40 (1.3)	6.91/6.93 (−0.3)
2.0	3.25/3.13 (3.5)	4.64/4.70 (−1.3)	5.69/5.73 (−0.7)	6.50/6.43 (1.1)
3.0	2.19/2.07 (5.7)	3.07/3.33 (−8.6)	3.85/3.90 (−1.3)	4.72/4.67 (1.1)

A temporal *planning estimate* (not a proven bound, and not an equality): if every template were re-queried in every freshness window, the number of (re-)noisings per template would be $N = \max(1, \lceil T/\tau \rceil) + T\lambda q$, giving the first-order estimate $\mathbb{E}[\varepsilon_{\text{temp}}] \approx \varepsilon_q \, \mathbb{E}[u_k] \, N$. Because N assumes every template is re-issued in *every* window—ignoring the actual arrival schedule—the estimate is conservative: across independently seeded trials it over-predicts the realized budget in every cell with active dynamics (at τ=10 it predicts ≈70 vs. a realized ≈38; at λ=0.2, ≈49 vs. ≈32; Section 4), so we commit to the *trend*, not the magnitude. We do *not* claim it as an upper bound—we have no theorem that $\mathbb{E}[\varepsilon_{\text{temp}}] \le \varepsilon_q \mathbb{E}[u_k] N$ holds for every arrival process—and the only hard ceiling remains naive composition, $k\varepsilon_q$. Three regimes slide continuously: *(a) static* ($\tau \to \infty, \lambda$=0, N=1) recovers Section 3.2; *(b) bounded staleness* (τ finite, λ=0); *(c) update-driven* (λ>0). The prototype is unit-tested across parser validation, sensitivity, Laplace calibration, cache semantics, the model limits, the concentration bound, and temporal re-noising.

4 Results and Findings

The single most important result is the real-trace forecast of Section 4.2—predicting a workload's full distinct-query count, hence its budget, from only its first half cuts the error by 45% over the plug-in estimate; the synthetic campaign below first establishes the model's arithmetic and invariances.

Setup. The campaign comprises **six sweeps totaling** 4,155 **trials** (~150,000 queries) over Zipf workloads (m=7 Adult age-bands and TPC-H templates), four per-query budgets $\varepsilon_q \in \{0.1, 0.5, 1, 2\}$, lengths $k \in \{10, \ldots, 500\}$, and two data scales (TPC-H SF=1 with 6M lineitem rows, SF=10 with 60M). Comparisons are between internal execution modes under an identical middleware, to isolate the algorithmic effect of workload-aware accounting; end-to-end comparison against external prototypes would require query-rewriting integration and is left as future work.

4.1 Validating the model

Model vs. empirical $\mathbb{E}[u_k]$. Table 2 compares the closed form to the 30-trial empirical mean across the main grid: agreement is within < 3% in 21 **of** 24 **cells**; the three outliers (3.5–8.6%) occur at high skew and small k, where $\mathbb{E}[u_k]$ is tiny and a 0.1-unit deviation is a large relative error. The savings ratio $S(k)$, computed from measured budget, matches within < 2% in 23/24 cells.

Invariance and limits. The model's $\mathbb{E}[u_k]$ is structurally independent of ε_q (confirmed within 0.12 units across $\varepsilon_q \in \{0.1, 0.5, 1, 2\}$) and of data scale: the same templates over a 10× larger TPC-H database (60M rows) give identical empirical $\mathbb{E}[u_k]$ to within 0.034 units, isolating workload structure from data scale. Pushing skew to α=10 at k=200, predicted and empirical $\mathbb{E}[u_k]$ stay within 0.13 units while $\mathbb{E}[u_k]$ varies by an order of magnitude, and $S(k) \to 99.4\%$ recovers the $1 - 1/k$ corner numerically (Figure 3).

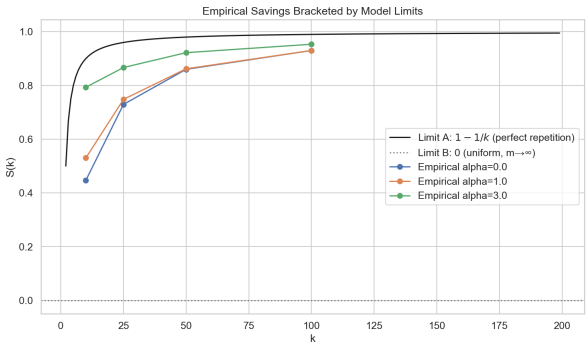

Figure 3: Savings ratio $S(k)$ vs. workload length, several Zipf α. Dashed: model. Markers: empirical mean (30 trials). High-α curves approach $1 - 1/k$ (Limit A); the α=0 curve stays near 0 until k approaches m, then saturates (Limit C, finite m).

Robustness. Four properties make the forecast dependable. *(i) Distribution-agnostic*: it is exact for any i.i.d. marginal (uniform, Zipf, lognormal-shaped), matching the empirical mean to ≤ 0.2. *(ii) Concentrated*: by Proposition 4 the standard deviation is small, $\sqrt{V}$=0.34–1.22 at m=20, so the spread around the forecast is tightly controlled—a variance- or Chebyshev-scale statement, not a high-probability envelope. *(iii) Exact on heterogeneous workloads*: on a constructed mixed W1+W4 workload with true $u_k = 1 + (1 - f)k$, the closed form $\varepsilon_q u_k$ matches the measured budget *to the cent* at every mixing fraction (Table 3). *(iv) Conservative under* positive *correlation*: a bursty sticky-Markov process (the tested case; same stationary marginal) clusters repeats and lowers the distinct count, so the i.i.d. forecast *over*-predicts u_k by 1.09/1.29/2.72× at stickiness s=0.3/0.6/0.9—the safe direction. We claim this only for the tested positive-correlation process: *anti*-correlated or without-replacement arrivals would instead push u_k *above* i.i.d. and make the forecast optimistic. The unconditional safety net is therefore the ε_q=B/m allocator: in the *static* regime $u_k \le m$ for any arrival process, so it never rejects (under temporal re-noising the same species can be re-charged, so this becomes a per-window guarantee).

Table 3: Mixed W1+W4 workload, B=50, ε_q=0.5, 20 trials. The closed form matches the measured budget exactly.

f	true u_k	measured ε used	predicted $\varepsilon_q u_k$
0.1	91	45.5	45.5
0.3	71	35.5	35.5
0.5	51	25.5	25.5
0.7	31	15.5	15.5
0.9	11	5.5	5.5

Temporal extension. The temporal model is validated by sweeping τ (with λ=0) and λ (with τ large), over independently seeded trials so each draws an independent update stream. The planning estimate $\varepsilon_q \mathbb{E}[u_k]N$ tracks the empirical mean in trend and matches at large τ (rare re-noising); in every cell with active dynamics it *over*-predicts the realized budget—at τ=10, ≈70 vs. an empirical ≈38; at λ=0.2, ≈49 vs. ≈32—since not all re-noisings materialize within the horizon. We therefore report it as a *conservative planning estimate*, not a proven bound. Freshness has an explicit, if conservatively-estimated, cost.

4.2 Real-workload validation (Redbench)

The synthetic sweep cannot establish *external* validity, so we add the first test on production-derived traces: Redbench [23], 30 analytical SQL workloads synthesized from Amazon Redshift's Redset query log, in which each user's real query timeline (and thus its repetition structure) is mapped onto CEB+/JOB query instances. Redbench is *not* a differential-privacy benchmark; we use its production-derived query-repetition structure solely to externally validate our workload-occupancy forecast, not any DP mechanism of its own. The exact-query identity is the SQL instance, so the realized distinct count u_k and Redbench's own *query-repetition rate* $= 1 - u_k/k$ are exactly our $S(k)$. Running the occupancy model against all 30 workloads yields four findings. *(1) Real workloads span the full skew range:* realized $S(k)$ runs from 0% to 98% (mean 53%), so template repetition—the structure the model exploits—is pervasive in production, not an artefact of the sweep. *(2) Real distributions are heavy-tailed:* fitting Zipf to each workload gives $\alpha \in [0, 3]$ with mean 0.84 (near classic Zipf), and our synthetic sweep $\alpha \in \{0, \ldots, 3\}$ *brackets* the real range—answering "why Zipf?" with production data. *(3) Descriptive fit:* fed the full-trace $\{p_i\}$, the closed form reproduces the realized savings to a mean absolute 0.13 in $S(k)$—but this is an *in-sample, descriptive* check (it uses the whole trace), not a before-execution forecast. *(4) The actual forecast* (our headline real-data result): estimating $\{p_i\}$ from only the *first* 50% of each timeline and predicting the full u_k, the smoothed Good–Toulmin estimator [20] cuts the plug-in's error by 45% (0.22 → 0.12 in u_k/k). *In plain terms*, the *plug-in* simply assumes the rest of the workload repeats the query mix seen so far, whereas the *Good–Toulmin* (unseen-species) estimator also predicts how many *brand-new* queries are still to come—the classic "how many species have we not yet seen?" problem—so it forecasts u_k more accurately from a short prefix. This—not the in-sample fit—is the genuine forecast, and it confirms on production-derived data that unseen-species correction is the right tool. Honestly, Redbench is trace-*derived* synthesis (real repetition structure, CEB+/JOB queries, not raw production SQL), so it raises external validity substantially without fully eliminating the gap (Section 6).

4.3 Budget savings and where naive composition fails

Full grid. Across six workloads, workload-aware accounting answers the same 100 queries at comparable per-query MAE while spending only $\varepsilon_q u_k$: savings of 100× on the repetitive W1, 33×, 20×, 14.3× on the TPC-H and uniform pools (where u_k saturates at the pool size m), and *exactly* 1× on the genuine drill-down W4, where every query is unique and the model correctly predicts $S(k)$=0 (Figure 4). These multipliers are accounting ratios $1/(1 - S(k))$ that any exact-repeat cache attains; our contribution is the closed-form *prediction* of $S(k)$ before execution (within < 3%), not the saving itself.

Table 4: Full grid (6 workloads, k=100, ε_q=1, 30 trials). Workload-aware spends $\varepsilon_q u_k$ at 93–99% cache-hit rates; the drill-down (W4) is the predicted no-win case.

Workload	naive ε	work.-aware ε	savings
W1 Repetitive	100.0	1.0	100×
W2 TPC-H returnflag	100.0	3.0	33×
W2 TPC-H priority	100.0	5.0	20×
W2 Zipf α=1	100.0	7.0	14.3×
W3 Uniform	100.0	7.0	14.3×
W4 Drill-down (unique)	100.0	100.0	1×

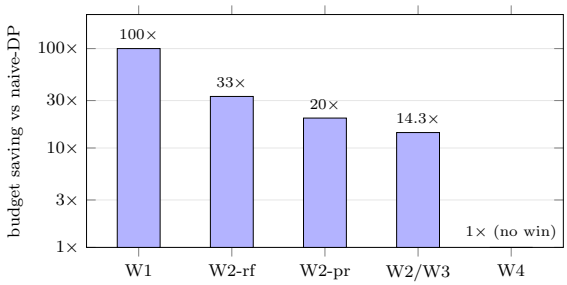

Figure 4: Budget saved vs. naive (k=100, ε_q=1, log scale): up to 100× on the repetitive W1 and 14–33× on the parametric/uniform pools, but exactly 1× (no win) on the unique drill-down W4—the model predicts the $S(k)$=0 case rather than overclaiming.

Two concrete failure regimes of naive composition. On the real middleware (Adult, B=10, k=100, 12 trials; Table 5), repetition turns the structural advantage into a stark gap. *(A) Budget exhaustion*: a dashboard re-issuing one tile at fixed ε_q=1 exhausts naive after B/ε_q=10 queries (10/100 answered), while caching answers all 100/100 for free—a 10× gap. *(B) Accuracy at fixed total budget*: to answer all 100, naive must thin the budget ($\varepsilon_q = B/k$=0.1, error ≈ 10), whereas concentrating it on the u_k=1 distinct release makes the answer essentially exact. *(C, honest counter)*: on a genuine drill-down both match at error ≈ 9.6—included precisely because the model predicts the no-win case.

Table 5: Naive's failure regimes vs. workload-aware (real middleware, Adult, B=10, k=100).

Workload	Naive ans.	Naive |err|	Workload-aware ans.	Workload-aware |err|
Repetitive, fixed ε_q=1	10/100	0.9	**100/100**	0.4
Repetitive, fixed budget B	100/100	10.0	100/100	≈ **0**
Drill-down (all unique)	100/100	9.6	100/100	9.6

4.4 Putting the model to work: predictive allocation

The model is descriptive and also *prescriptive*: an allocator estimates $\hat{U} = \mathbb{E}[u_k]$ online (after a short warmup) and sets $\varepsilon_q = B/\hat{U}$ to target the offline optimum $\varepsilon_q^* = B/u_k$ *under a uniform per-distinct-release objective* (a frequency-/importance-weighted error tilts ε toward the frequent templates—Appendix C), capping spend at B. At a fixed *total* budget this trades coverage for accuracy: on Zipf workloads (k=100, B=10) it lowers per-query MAE on the *answered* subset (Table 6; it answers 69–88/100 where naive answers 100) by 16.8% (α=0) down to 5.4% (α=2). We are deliberately careful: the gain is only *borderline* (paired t-test $p \approx 0.05$ at α=0, not crossing 0.05; $p > 0.3$ for $\alpha \geq 1$)—directional, not a uniform win, and bought with late-query rejections. The robust result is instead the *safe* allocator ε_q=B/m, which never rejects in the static regime ($u_k \leq m$; temporal re-noising can re-charge a species) and matches a learned bandit's best operating point at zero exploration cost; the budget-sweep (32% at B=1) and scale-invariance (40% at SF=10) details are in Appendix A.

Table 6: Predictive allocator vs. fixed-ε_q baselines (mean±SE, 30 trials, k=100, B=10). MAE is over answered queries; the allocator answers fewer (69–88) than naive (100). The gain is borderline ($p \approx 0.05$) only at α=0.

α	Mode	MAE	answered	ε spent
0.0	Naive	9.86	100/100	10.0
	Predictive	**8.21**±0.85	69/100	10.0
1.0	Naive	9.81	100/100	10.0
	Predictive	**8.98**±0.93	78/100	10.0
2.0	Naive	9.96	100/100	10.0
	Predictive	**9.42**±1.40	88/100	10.0

Where we lead, and where we are merely on par. We frame the comparison as concrete situations a custodian faces, and are honest that on the raw numbers we are level, not ahead. *What the forecast adds, before execution:* a t=0 completion check (a hard yes under B/m, since $u_k \leq m$) and a *per-release* (α, β) accuracy verdict (Appendix B), plus capacity planning from a prefix (the 45% Redbench result); reactive systems surface these only by spending budget query-by-query. *Where we do not lead:* on the savings *number* we cite the baselines' own published figures rather than re-running them—Turbo 1.7–15.9× [13], CacheDP a reduction under an accuracy contract [14]—against our 14–33× on the pools and 100× on pure repetition: the same order of magnitude, so we claim *parity*, not superiority (the workloads and mechanisms differ, so this is a ballpark placement, not a controlled run). At a fixed budget the safe ε_q=B/m allocator simply *reaches* a tuned ε-greedy bandit's best operating point at zero exploration cost (MAE 2.0 vs. 3.8 at the same answer rate; Appendix A). The contribution is the *before-execution* deliverable the others lack, not a larger multiplier.

4.5 Privacy leakage

We complement the budget/utility analysis with two attacks that confirm the per-query calibration. *Membership inference*: an adversary observing one noisy COUNT must decide whether a target is present. The optimal single-release attacker's *accuracy* is bounded by $e^\varepsilon/(1+e^\varepsilon)$; the empirical AUC—which for this symmetric single-release threshold test nearly equals the attacker's accuracy—stays at or below that bound (within $\leq 1\%$) and is at chance for $\varepsilon \leq 0.1$ (Table 7, left). A fully type-matched comparison would use the exact Laplace AUC; the two nearly coincide here. *Reconstruction*: replaying a 5-level differencing drill-down on real Adult counts (48842, 34327, 24137, 7210, 3137), the mean absolute error scales as $1.5/\varepsilon$ (the expected difference of two Lap$(1/\varepsilon)$ variables) to within a few percent across three orders of magnitude (Table 7, right, indexed by the *per-release* ε). The attack is *five* releases, so by sequential composition it costs a *cumulative* 5ε; at a fixed total budget B the per-release value is ε=$B/5$ and the error scales accordingly—the Dinur–Nissim view ties reconstruction to the total budget spent, not the per-query value. A stronger shadow-model MIA (a classifier trained on simulated middleware runs) is harder to read, and we are careful not to overclaim it: across four workloads × four ε_q (32 cells) with only 30 shadow runs per world and an 18-sequence single held-out split, the AUC ranges 0.14–0.58. An AUC below 0.5 flips above 0.5 under score-direction reversal, so with an 18-sample test it signals small-sample estimator variance and a possibly ill-oriented classifier, *not* sub-chance leakage. The honest reading is that this setup cannot establish *strong* workload-level leakage; a rigorous claim would need repeated stratified cross-validation, confidence intervals, and validation-set score-direction selection (future work). Our calibration conclusion therefore rests on the clean single-query MIA and the reconstruction above, both of which degrade exactly where the model predicts efficient budgeting (high skew, low ε_q); the reconstruction in particular succeeds on the W4 drill-down, which the model flags as the no-savings case.

Table 7: Leakage vs. per-release ε. Left: single-release membership-inference AUC vs. the DP *accuracy* bound $e^\varepsilon/(1+e^\varepsilon)$ (AUC≈accuracy for this symmetric test). Right: differencing reconstruction error vs. the $1.5/\varepsilon$ reference (a 5-release attack; cumulative cost 5ε).

ε	AUC	$\frac{e^\varepsilon}{1+e^\varepsilon}$	ε	rec. err	$1.5/\varepsilon$
0.01	0.499	0.503	0.01	148.2	150.0
0.10	0.522	0.525	0.10	14.8	15.0
1.00	0.724	0.731	1.00	1.5	1.5
5.00	0.988	0.993	5.00	0.3	0.3

4.6 Runtime and overhead

The privacy machinery is cheap relative to I/O. Instrumenting 1,500 queries, cache hits are 2.7× faster than misses (0.76 vs. 2.02 ms) because they skip the database round-trip (36% of miss cost); SQL parsing is 21%, and the privacy-specific stages (sensitivity, allocation, noise) sum to under 0.25 ms. Tail latency is controlled (miss p99=5.1 ms, hit p99=3.6 ms), and single-threaded throughput is ∼495 q/s on the miss path and ∼1,320 q/s on the hit path, so a workload-aware mode at 93–99% hit rate sustains analyst-tier load. The asymptotic cost of the forecast itself is $O(m)$ per evaluation of $\mathbb{E}[u_k]$.

Figure 5: Shadow-model MIA AUC across W1–W4 at $\varepsilon_q \in \{0.1, 0.5, 1, 2\}$ (32 cells, 18-sequence single test split). AUC in $[0.14, 0.58]$; we read this as *inconclusive* (small-sample, single split: AUC$<$0.5 flips under score reversal), not as established weak leakage. Repeated CV and CIs are future work.

Interpretation. The two validations carry different weight: the synthetic grid (the $< 3\%$ agreement) is a *self-consistency* check—it confirms the estimator's arithmetic, not that real workloads are i.i.d. Zipf—whereas the genuine external evidence is the Redbench prefix forecast, a 45% error reduction on traces the model never saw.

5 Future Work

Several extensions follow directly. *(1) Raw production traces:* the Redbench validation uses trace-derived synthesis; running the model on raw analyst query logs (and executing them through a DP engine end-to-end) would close the remaining external-validity gap. *(2) Tight composition:* replacing basic sequential composition with Rényi DP [16] or zCDP [17] would predict $\sum \varepsilon_i$ rather than count u_k, with the closed-form structure mostly unchanged, and composes with the caching (tighter accounting lowers the per-release rate; caching lowers the number of paid releases—orthogonal axes). *(3) Multi-table / JOINs:* the occupancy framework generalizes to JOIN-shaped queries given a per-template sensitivity bound such as residual sensitivity [25] or R2T [26]. *(4) Equivalence-checked semantic cache:* pairing tree-kernel/embedding similarity with a symbolic equivalence prover (predicate normalization, range merging, α-conversion) would let the cache safely admit non-identical-but-equivalent queries (Appendix D). *(5) Cache re-release and AVG decomposition:* re-noising stale cache entries on budget surplus, and a SUM+COUNT decomposition of AVG with per-group accounting, would tighten utility. *(6) Private group-key domains:* a stability-based threshold that suppresses low-count groups would lift the public-key-domain assumption at a small δ cost. *(7) A full temporal model:* the staleness/update extension of Section 3.3 is a first-order planning estimate; a proper arrival-process treatment with re-release policies on budget surplus deserves to be developed and validated as a study of its own.

6 Discussion and Conclusion

Findings. (1) Budget consumption is a function of $\mathbb{E}[u_k]$, not of the cache implementation: $\mathbb{E}[\varepsilon_{\text{wa}}] = \varepsilon_q \mathbb{E}[u_k]$ holds within $< 3\%$ in 21/24 cells, exactly on the mixed workload, and is invariant to scale and ε_q—so a deployment can estimate its privacy cost *before* running a query. (2) The $1 - 1/k$ rule is the high-skew corner of one model. (3) The per-query calibration is clean—single-query MIA stays at or below the $e^\varepsilon/(1+e^\varepsilon)$ accuracy bound (within 1% for $\varepsilon \le 1$) and reconstruction tracks $1.5/\varepsilon$—while the workload-level shadow MIA was inconclusive at our sample size (an honest negative on the experiment, not a claim). (4) The model is honest about failure: on *unique* workloads (and uniform over a large/growing support) savings vanish and reconstruction succeeds, exactly as predicted; on a small finite pool, uniform still saturates at $m\varepsilon_q$ and saves.

Practical implication. Because $\mathbb{E}[u_k]$ is computable before any query runs and is invariant to data scale and ε_q, the custodian of the dashboard scenario can size ε_q *a priori*: the safe choice $\varepsilon_q = B/m$ provably finishes the month in the static regime (since $u_k \le m$; under temporal re-noising this becomes a per-window guarantee), and the predictive allocator pushes accuracy further when the workload is skewed. The model also tells the custodian when caching cannot help—a drill-down workload spends like naive composition—so the privacy/utility budget can be planned, not discovered after exhaustion. At a *fixed* ε_q, budgeting and attack-resistance improve together in the high-skew regime; this is not automatic, though—if the allocator exploits skew to *raise* ε_q (e.g. B/u_k with small u_k), per-release leakage rises, so the planner must weigh the two.

Threats to validity. We separate three. *Construct*: the headline "$\mathbb{E}[u_k]$ within $< 3\%$" is a Monte-Carlo *self-consistency* check—the empirical u_k is sampled from the very Zipf distribution the closed form integrates, so the mean converges to $\mathbb{E}[u_k]$ *by construction*; it validates the estimator's arithmetic, not the claim that real workloads are i.i.d. Zipf. *Internal*: the forecast assumes a non-adaptive analyst; against a worst-case adaptive analyst the only valid bound is $u_k \le \min(k, m)$, i.e. the ε_q=B/m safe allocator. *External*: the headline grid is synthetic, but we now validate on Redbench (Section 4.2)—30 production-*derived* traces spanning 0–98% repetition with fitted Zipf $\alpha \in [0, 3]$ (mean 0.84)—which raises external validity substantially. The residual gap is honest: Redbench is trace-*derived* synthesis (real repetition structure, benchmark queries rather than raw production SQL), and the i.i.d. closed form is mildly over-optimistic about savings at the low-repetition end. Scope is single-table aggregates under basic composition; JOINs and tight (Rényi/zCDP) composition are future work.

Conclusion and reproducibility. A closed-form model parameterized by $(\{p_i\}, k)$ predicts realized DP-budget consumption in repeated aggregate SQL workloads, recovers the folklore $1 - 1/k$ rule as a corner, and is validated to $< 3\%$ across 4,155 trials—while being explicit about the regimes in which it predicts no benefit. The middleware, an interactive bilingual (EN/TR) web demo that animates the live pipeline step by step, and all experiment generators are public[1] with fixed integer seeds, so every table is reproduced by re-running its generator (a canonical subset of result CSVs is committed; the rest regenerate deterministically). Dependencies are listed in a `pyproject.toml` (minimum versions only; we do not ship an exact lockfile), so the model-validation and allocation experiments are deterministic within a fixed environment but not guaranteed bit-identical across library upgrades; the large six-sweep campaign is in any case only *distributionally* reproducible.

[1] https://github.com/canoztas/cmp653-project

Appendices

These appendices collect material that supports but goes beyond the core paper: the predictive allocator and its learned-allocator comparison (A), a-priori feasibility planning (B), a blind spot of assignment-free reward proxies (C), the negative result on semantic caching (D), a glossary (E), and an interactive web demo (F). The complete implementation, the web demo, and every experiment generator are public at https://github.com/canoztas/cmp653-project; the datasets are the standard public TPC-H, UCI Adult, and Redbench benchmarks.

A Predictive Budget Allocator

The model is also *prescriptive*: instead of fixing ε_q offline, an allocator can estimate $\hat{U} = \mathbb{E}[u_k]$ online from the empirical (Laplace-smoothed) template distribution after a short warmup and set $\varepsilon_q = B/\max(\hat{U}, 1)$, capped to the remaining budget so total spend never exceeds B. This targets $\varepsilon_q^* = B/u_k$, the offline optimum *under a uniform per-distinct-release objective* (Proposition 2; under a frequency- or importance-weighted error the optimum tilts ε toward the frequent/important templates, Appendix C). On Zipf workloads (k=100, B=10, 30 trials) the predictive allocator lowers per-query MAE (over answered queries) by up to ∼17% at low skew (point estimates 16.8% at α=0 down to 5.4% at α=2, and up to 32% in the tightest-budget regime B=1), but the gain is only *borderline* (paired t-test $p \approx 0.05$ at α=0, not crossing the 0.05 threshold; $p > 0.3$ for $\alpha \geq 1$); we report it as directional, not a uniform win. The regime-independent result is instead the *safe* allocator ε_q=B/m: since $u_k \leq m$ in the static regime, it never rejects and needs no warmup, and on the benchmark it answers all queries at fresh-release MAE 2.0 versus an ε-greedy bandit's 3.8 at the same 100% rate—a 1.9× gap that is the bandit's exploration tax, since B/m *is* the bandit's best-arm operating point (holding when $m \leq k/4$).

Table 8: Online allocation policies (Zipf, m=20, k=100, B=10, 60 trials). Fresh MAE (over cache misses) is reported jointly with the answered fraction.

Policy	Fresh MAE	Answered	ε used
Naive (ε_q=B/k)	10.1	100/100	1.6
ε-greedy bandit	3.8	100/100	6.1
Safe closed form (B/m)	**2.0**	**100/100**	8.0
Oracle (B/u_k, needs forecast)	1.6	100/100	10.0

The allocator's quality hinges on predicting u_k from a warmup prefix. The default plug-in occupancy estimate uses only observed templates and so under-predicts; horizon-aware extrapolators do better in the realistic regime where the warmup is at least a third of the workload ($t = (k-n)/n \leq 2$): the Good–Toulmin family roughly halves the error, and the smoothed variant [20] tames its divergence past the $t \leq 1$ validity limit (Table 9). Chao1 [21] estimates total richness, not the horizon-k count, and over-predicts. Better u_k accuracy is a coverage/noise trade-off, not a free win: smoothed-GT lifts the answered fraction from 81% to 92% at higher per-release noise.

Table 9: u_k-prediction MAE (%) by estimator and extrapolation factor t (Zipf, m=50, k=100). Lower is better.

estimator	t=0.5	t=1	t=2	t=4
Plug-in occupancy	18	30	43	58
Good–Toulmin [19]	**8**	**17**	193	204
Smoothed GT [20]	**8**	**17**	**27**	151
Chao1 [21]	55	48	47	47

B A-Priori Feasibility Planning

The forecast also supports *capacity planning*: from a declared template distribution $\{p_i\}$ and horizon k, the planner can size ε_q and emit a feasibility verdict *before any query runs and before any budget is spent*. Using the safe sizing $\varepsilon_q = B/m$ (static-regime completion guaranteed since $u_k \leq m$), the per-release accuracy is governed by $\mathbb{P}[\,|\eta| \leq \alpha\,] = 1 - e^{-\alpha\varepsilon_q/\Delta f}$, so the planner emits a *per-release* (α, β) verdict—"does *each* release *individually* meet |noisy − true| $\leq \alpha$ with probability $\geq 1-\beta$?" (the standard per-query (α,β)-accuracy notion)—as PASS iff $1 - e^{-\alpha\varepsilon_q/\Delta f} \geq 1-\beta$. We are careful that this is *not* the much stronger joint event that *all* releases succeed at once (which would need the product $p^{u_k} \geq 1-\beta$—e.g. $\alpha \approx 9$ here—since under independence the all-succeed probability is the *product* of per-release successes, not a union bound). The experiment measures exactly the per-release reading—the *fraction* of releases meeting the SLA, not the all-succeed event; on a Zipf workload (m=20, k=100, B=10, β=0.2, 200 trials) the t=0 verdict matches that realized fraction (FAIL at α=2, PASS at $\alpha \geq 4$), at 100% completion. No deployed system emits *this* verdict at t=0: reactive remaining-budget estimators are undefined at zero history, and accuracy-aware explorers such as APEx [8]—which *do* take a per-query accuracy target and return the minimum ε that meets it—decide one query at a time, reactively, with no a-priori workload-*completion* verdict. Our novelty is exactly that narrow combination: a workload-completion feasibility verdict computed at t=0 from the public support, before any budget is spent. We keep the honest boundaries: under an *under-declared* support a reactive estimator self-corrects and out-answers us, under an *unbounded* horizon a position-decaying schedule (Bai) outlasts us, and with no repetition the plan collapses to naive—all three are printed alongside the win.

C A Blind Spot of Assignment-Free Reward Proxies

In DP-SQL the per-query error is exactly what privacy hides, so a reward-driven allocator cannot observe its true objective. Consider the weakest common surrogate—an *assignment-free* reward that sees only the sorted granted-ε multiset and the rejection count, blind to which template each ε backs. On a weighted dashboard (one KPI at weight 10, three tiles at weight 1), two allocations with an *identical* such proxy differ by 2.71× in realized weighted error on the live middleware (0.188 vs. 0.509, 400 trials), so a learner restricted to that proxy cannot prefer the good one. We do *not* overclaim "allocation cannot be learned": a *contextual* bandit that sees the query identity and the public weights is not blind—it can tilt ε exactly as our closed form does ($\varepsilon_i \propto \sqrt{w_i}$). The narrow point is that the information closing the gap is the *public weights*, so a learner must be *given* the same public structure as context—computing the

allocation, not discovering it from a reward—and under flat weights the gap vanishes.

D A Negative Result: Semantic Caching

We tested whether structurally similar queries the exact cache misses could be matched via a Collins–Duffy tree kernel [24] plus an AST sentence-embedding (cosine), admitting a match only when both scores exceed conservative thresholds. The semantic cache fails in *both* directions: it admits false positives (queries differing only in WHERE literals have near-identical ASTs but different true answers, so a reused cached value is wrong by thousands of count units), and it misses textbook equivalences (commutative AND reordering, `age>=30` vs. `age>29`), because ordered-child kernels and text embeddings do not encode logical equivalence. Returning a cached answer for a non-equivalent query does *not* break privacy—reusing a DP output is still post-processing and costs zero extra budget regardless of which query it is returned for—but it is simply *wrong*: the answer belongs to a different query. The failure is one of *correctness*, not privacy. The lesson: tree-kernel/embedding similarity is not equivalence; a safe semantic cache needs a symbolic equivalence prover (predicate normalization, range merging, α-conversion), which we leave as future work.

E Glossary

Species / u_k: the budget-spending unit is the *exact* query—the structural template *together with* its WHERE literals, i.e. the cache key. u_k is the count of distinct such exact queries in a length-k workload, and $\{p_i\}$ is the distribution over them. Two queries that share a structural template but differ in literals are *distinct* species and each is charged. **Occupancy model:** $\mathbb{E}[u_k] = \sum_i [1-(1-p_i)^k]$, the expected number of non-empty cells when k balls fall into m cells with probabilities $\{p_i\}$. **Savings $S(k)$:** fractional budget saved versus naive composition, $1 - \mathbb{E}[u_k]/k$. **Parallel composition:** a GROUP BY costs ε_q, not $g\varepsilon_q$, because each tuple affects one group. **Post-processing:** any function of a DP output is DP at no extra cost (cache hits and top-k selection). **Safe allocator B/m:** per-query budget that never rejects in the static regime because $u_k \le m$ (temporal re-noising can re-charge a species).

F Interactive Web Demo

The artifact includes an interactive web demo (Flask) that runs each query through the *real* middleware over the real Adult table and animates the exact steps of `DPMiddleware.execute()`—parse/validate, template and parameter hashing, cache lookup (L1 exact, optional L2 semantic), the temporal clock tick, sensitivity, budget allocation, the Laplace mechanism, and cache write. A live panel tracks the budget ledger (spent/total), true-vs-released values, and the workload state (u_k, savings $S(k)$, predicted $\hat{U}$, the temporal clock). All six execution modes and the W1/W2/W4 presets are selectable, so one can watch repeats served free, the budget deplete on a drill-down, and the predictive allocator adapt ε_q=$B/\hat{U}$ in flight; every number, hash, and SQL string comes from the real backend, with bilingual (EN/TR) step explanations and a glossary mirroring Appendix E. The demo is in the repository (run `python demo/app.py`; URL in Section 6).

References

[1] C. Dwork, F. McSherry, K. Nissim, A. Smith. Calibrating Noise to Sensitivity in Private Data Analysis. *TCC*, 2006.
[2] C. Dwork, A. Roth. The Algorithmic Foundations of Differential Privacy. *Found. Trends TCS*, 2014.
[3] I. Dinur, K. Nissim. Revealing Information While Preserving Privacy. *PODS*, 2003.
[4] F. McSherry. Privacy Integrated Queries (PINQ). *SIGMOD*, 2009.
[5] I. Kotsogiannis et al. PrivateSQL: A Differentially Private SQL Query Engine. *PVLDB* 12(11), 2019.
[6] N. Johnson, J. P. Near, J. M. Hellerstein, D. Song. Chorus: a Programming Framework for Building Scalable Differential Privacy Mechanisms. *IEEE EuroS&P*, 2020.
[7] Y. Yu et al. DOP-SQL: Differentially Private Optimization for SQL. 2024.
[8] C. Ge, X. He, I. F. Ilyas, A. Machanavajjhala. APEx: Accuracy-Aware Differentially Private Data Exploration. *SIGMOD*, 2019.
[9] X. Zhang et al. BGTplanner: Maximizing Training Accuracy for Differentially Private Federated Recommenders via Strategic Privacy Budget Allocation. *IEEE Trans. Services Computing* 18(6), 2025, 3523–3531.
[10] C. Li, G. Miklau, M. Hay, A. McGregor, V. Rastogi. The Matrix Mechanism: Optimizing Linear Counting Queries under Differential Privacy. *VLDB Journal* 24(6), 2015.
[11] W. Dong, J. Sun, K. Yi. Better than Composition: How to Answer Multiple Relational Queries under Differential Privacy. *SIGMOD*, 2023.
[12] D. Pujol, A. Sun, B. Fain, A. Machanavajjhala. Multi-Analyst Differential Privacy for Online Query Answering. *PVLDB* 16(4), 2022.
[13] K. Kostopoulou, P. Tholoniat, A. Cidon, R. Geambasu, M. Lécuyer. Turbo: Effective Caching in Differentially-Private Databases. *SOSP*, 2023.
[14] M. Mazmudar, T. Humphries, et al. Cache Me If You Can: Accuracy-Aware Inference Engine for Differentially Private Data Exploration. *PVLDB* 16(4), 2022.
[15] M. Abadi et al. Deep Learning with Differential Privacy. *CCS*, 2016.
[16] I. Mironov. Rényi Differential Privacy. *CSF*, 2017.
[17] M. Bun, T. Steinke. Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds. *TCC*, 2016.
[18] P. Flajolet, D. Gardy, L. Thimonier. Birthday Paradox, Coupon Collectors, Caching Algorithms and Self-Organizing Search. *Discrete Applied Math.* 39(3), 1992.
[19] I. J. Good, G. H. Toulmin. The Number of New Species, and the Increase in Population Coverage, when a Sample is Increased. *Biometrika* 43, 1956.
[20] A. Orlitsky, A. T. Suresh, Y. Wu. Optimal Prediction of the Number of Unseen Species. *PNAS* 113(47), 2016.
[21] A. Chao. Nonparametric Estimation of the Number of Classes in a Population. *Scand. J. Statistics* 11(4), 1984.
[22] L. Ma et al. Query-based Workload Forecasting for Self-Driving Database Management Systems (QueryBot 5000). *SIGMOD*, 2018.
[23] S. Krid, M. Stoian, A. Kipf. Redbench: A Benchmark Reflecting Real Workloads. *aiDM @ SIGMOD*, 2025. arXiv:2506.12488. (Workloads synthesized from Amazon Redshift's Redset query traces.)
[24] M. Collins, N. Duffy. Convolution Kernels for Natural Language. *NeurIPS*, 2001.
[25] W. Dong, K. Yi. Residual Sensitivity for Differentially Private Multi-Way Joins. *SIGMOD*, 2021.
[26] W. Dong, J. Fang, K. Yi, et al. R2T: Instance-optimal Truncation for Differentially Private Query Evaluation with Foreign Keys. *SIGMOD*, 2022.
[27] P. Mundra, A. Sealfon, Z. Sun, Q. C. Liu. LAPRAS: Learning-Augmented Private Answering for Linear Query Streams. arXiv:2605.01960, 2026.
[28] S. Peng, Y. Yang, Z. Zhang, M. Winslett, Y. Yu. Query Optimization for Differentially Private Data Management Systems (Pioneer). *ICDE*, 2013.
[29] J. Acharya, G. Kamath, Z. Sun, H. Zhang. INSPECTRE: Privately Estimating the Unseen. *ICML*, 2018.